ЛАБОРАТОРНАЯ РАБОТА 2

ОБРАБОТКА ПОТОКОВЫХ ДАННЫХ

1. Развернуть локальный кластер Kafka (Zookeeper + Broker)

Для построения надежной инфраструктуры потоковой передачи данных был развернут локальный кластер Apache Kafka совместно с координатором Zookeeper с использованием Docker Compose.

Контейнеры успешно запущены. Внутри брокера `kafka-lab2` был создан топик `events` (с 3 партициями для распараллеливания) для приема данных от сенсоров:

```
docker exec kafka-lab2 kafka-topics --create --topic events --bootstrap-server
localhost:9092 --partitions 3 --replication-factor 1
```

```
PS D:\VSCode\Python\BSU\AODB\Lab2> docker ps
CONTAINER ID   IMAGE                             COMMAND                  CREATED         STATUS         PORTS
    NAMES
56a96564f5ab   confluentinc/cp-kafka:7.3.2       "/etc/confluent/dock…"   6 seconds ago   Up 5 seconds   0.0.0.0:9092->9092/tcp
    kafka-lab2
e14bdb0a71db   confluentinc/cp-zookeeper:7.3.2   "/etc/confluent/dock…"   6 seconds ago   Up 5 seconds   2888/tcp, 0.0.0.0:2181->2181/tcp, 3888/tc
p   zookeeper-lab2
PS D:\VSCode\Python\BSU\AODB\Lab2>
```

---

2. Написать продюсера на Python для имитации потока событий

Для генерации непрерывного потока данных был написан Python-скрипт `producer.py` с использованием библиотеки `kafka-python`. Скрипт имитирует работу датчиков температуры, бесконечно отправляя в топик `events` JSON-сообщения со скоростью 2 сообщения в секунду. Каждое сообщение содержит метку времени генерации `timestamp`.

Скрипт успешно запущен и непрерывно отправляет поток данных в Kafka:

```
PS D:\VSCode\Python\BSU\AODB\Lab2> python producer.py
Starting event generation
D:\VSCode\Python\BSU\AODB\Lab2\producer.py:20: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future
version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  event_time = datetime.utcnow().isoformat() + "Z"
Sent: {'sensor_id': 'sensor_3', 'temp': 23.44, 'timestamp': '2026-10-26T16:07:11.318338Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 26.94, 'timestamp': '2026-10-26T16:07:11.953835Z'}
Sent: {'sensor_id': 'sensor_1', 'temp': 21.89, 'timestamp': '2026-10-26T16:07:12.454982Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 27.92, 'timestamp': '2026-10-26T16:07:12.956250Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 26.31, 'timestamp': '2026-10-26T16:07:13.457342Z'}
Sent: {'sensor_id': 'sensor_3', 'temp': 23.54, 'timestamp': '2026-10-26T16:07:13.958954Z'}
Sent: {'sensor_id': 'sensor_1', 'temp': 25.2, 'timestamp': '2026-10-26T16:07:14.460295Z'}
Sent: {'sensor_id': 'sensor_1', 'temp': 23.73, 'timestamp': '2026-10-26T16:07:14.961484Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 23.08, 'timestamp': '2026-10-26T16:07:15.462934Z'}
Sent: {'sensor_id': 'sensor_3', 'temp': 24.56, 'timestamp': '2026-10-26T16:07:15.964160Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 24.95, 'timestamp': '2026-10-26T16:07:16.466221Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 22.71, 'timestamp': '2026-10-26T16:07:16.968302Z'}
Sent: {'sensor_id': 'sensor_1', 'temp': 24.1, 'timestamp': '2026-10-26T16:07:17.469732Z'}
Sent: {'sensor_id': 'sensor_1', 'temp': 22.91, 'timestamp': '2026-10-26T16:07:17.971246Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 20.5, 'timestamp': '2026-10-26T16:07:18.473061Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 20.95, 'timestamp': '2026-10-26T16:07:18.975680Z'}
Sent: {'sensor_id': 'sensor_3', 'temp': 22.0, 'timestamp': '2026-10-26T16:07:19.476378Z'}
Sent: {'sensor_id': 'sensor_2', 'temp': 29.89, 'timestamp': '2026-10-26T16:07:19.978084Z'}
Sent: {'sensor_id': 'sensor_1', 'temp': 22.02, 'timestamp': '2026-10-26T16:07:20.479513Z'}
```

---

3. Реализация стрим-приложения на Spark Structured Streaming

Для потоковой обработки событий из Kafka было написано приложение на Python с использованием API Spark Structured Streaming.

Приложение считывает сырые байты из топика `events`, парсит JSON-строки в типизированные колонки (`sensor_id`, `temp`, `timestamp`) и применяет оконные агрегации.

---

4. Обработка Event-Time, Окна и Состояние

В Spark Structured Streaming была реализована концепция Event Time — время, когда событие фактически произошло на датчике, а не когда оно дошло до кластера.

Для корректной работы с задержанными (late) событиями и очистки оперативной памяти (управления состоянием State Store) применена функция Watermarking со значением `10 seconds`. Spark поддерживает состояние незаконченных окон в оперативной памяти и закрывает окно только тогда, когда максимальное встреченное время события превысит конец окна + 10 секунд задержки.

Были протестированы три типа оконных функций:

1. Фиксированные окна

Окна имеют фиксированный размер (1 минута) и не пересекаются. Каждое событие попадает строго в одно окно.

```
-------------------------------------------
Batch: 1
-------------------------------------------
+------------------------------------------+---------+-----+
|window                                    |sensor_id|count|
+------------------------------------------+---------+-----+
|{2026-10-26 16:36:00, 2026-10-26 16:37:00}|sensor_3 |16   |
|{2026-10-26 16:36:00, 2026-10-26 16:37:00}|sensor_1 |6    |
|{2026-10-26 16:36:00, 2026-10-26 16:37:00}|sensor_2 |8    |
+------------------------------------------+---------+-----+

26/10/26 16:36:41 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 10000 milliseconds, but spe
nt 11164 milliseconds
-------------------------------------------
Batch: 2
-------------------------------------------
+------------------------------------------+---------+-----+
|window                                    |sensor_id|count|
+------------------------------------------+---------+-----+
|{2026-10-26 16:36:00, 2026-10-26 16:37:00}|sensor_3 |25   |
|{2026-10-26 16:36:00, 2026-10-26 16:37:00}|sensor_1 |11   |
|{2026-10-26 16:36:00, 2026-10-26 16:37:00}|sensor_2 |16   |
+------------------------------------------+---------+-----+
```

На скриншоте видно, что счетчик `count` накапливает количество событий для каждого датчика строго в пределах изолированных минутных интервалов (например, с 16:36:00 до 16:37:00).

2. Скользящие окна

Окно размером 1 минута, которое сдвигается каждые 30 секунд. В этом случае окна пересекаются, и одно событие участвует в агрегации сразу нескольких окон.

```
Batch: 1
-------------------------------------------
+------------------------------------------+---------+-----+
|window                                    |sensor_id|count|
+------------------------------------------+---------+-----+
|{2026-10-26 16:37:00, 2026-10-26 16:38:00}|sensor_1 |8    |
|{2026-10-26 16:37:00, 2026-10-26 16:38:00}|sensor_3 |8    |
|{2026-10-26 16:37:30, 2026-10-26 16:38:30}|sensor_3 |12   |
|{2026-10-26 16:38:00, 2026-10-26 16:39:00}|sensor_2 |3    |
|{2026-10-26 16:37:00, 2026-10-26 16:38:00}|sensor_2 |8    |
|{2026-10-26 16:37:30, 2026-10-26 16:38:30}|sensor_1 |12   |
|{2026-10-26 16:38:00, 2026-10-26 16:39:00}|sensor_3 |4    |
|{2026-10-26 16:38:00, 2026-10-26 16:39:00}|sensor_1 |4    |
|{2026-10-26 16:37:30, 2026-10-26 16:38:30}|sensor_2 |11   |
+------------------------------------------+---------+-----+

26/10/26 16:38:16 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 10000 milliseconds, bu
nt 11338 milliseconds
-------------------------------------------
Batch: 2
-------------------------------------------
+------------------------------------------+---------+-----+
|window                                    |sensor_id|count|
+------------------------------------------+---------+-----+
|{2026-10-26 16:37:30, 2026-10-26 16:38:30}|sensor_3 |21   |
|{2026-10-26 16:38:00, 2026-10-26 16:39:00}|sensor_2 |10   |
|{2026-10-26 16:37:30, 2026-10-26 16:38:30}|sensor_1 |18   |
|{2026-10-26 16:38:00, 2026-10-26 16:39:00}|sensor_3 |13   |
|{2026-10-26 16:38:00, 2026-10-26 16:39:00}|sensor_1 |10   |
|{2026-10-26 16:37:30, 2026-10-26 16:38:30}|sensor_2 |18   |
+------------------------------------------+---------+-----+
```

На скриншоте зафиксировано пересечение интервалов: сформированы окна `16:37:00-16:38:00` и `16:37:30-16:38:30`. Данные, пришедшие в 16:37:45, зачтутся в счетчики обоих окон.

3. Сеансовые окна

Окна с динамическим размером, зависящим от активности источника данных. Сессия расширяется до тех пор, пока поступают новые события. Если от датчика нет данных в течение заданного порога неактивности (timeout = 30 секунд), сессия закрывается. Так как размер окна динамический, Spark использует режим вывода `Complete`.

```
-------------------------------------------
Batch: 1
-------------------------------------------
+------------------------------------------------------------+---------+-----+
|session_window                                              |sensor_id|count|
+------------------------------------------------------------+---------+-----+
|{2026-10-26 16:41:59.161874, 2026-10-26 16:42:39.189355}|sensor_1 |4    |
|{2026-10-26 16:41:57.658207, 2026-10-26 16:42:39.690593}|sensor_2 |14   |
|{2026-10-26 16:42:00.666255, 2026-10-26 16:42:38.688146}|sensor_3 |7    |
+------------------------------------------------------------+---------+-----+

26/10/26 16:42:21 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interv
nt 11453 milliseconds
-------------------------------------------
Batch: 2
-------------------------------------------
+------------------------------------------------------------+---------+-----+
|session_window                                              |sensor_id|count|
+------------------------------------------------------------+---------+-----+
|{2026-10-26 16:41:59.161874, 2026-10-26 16:42:49.720647}|sensor_1 |11   |
|{2026-10-26 16:41:57.658207, 2026-10-26 16:42:50.222028}|sensor_2 |24   |
|{2026-10-26 16:42:00.666255, 2026-10-26 16:42:51.225137}|sensor_3 |13   |
+------------------------------------------------------------+---------+-----+
```

На скриншоте видно динамическое время начала и конца сессии в колонке `session_window`. С каждым новым микро-батчем время конца сессии отодвигается вперед, а счетчик событий инкрементируется.

---

5. Реализация стрим-приложения на Apache Flink

В отличие от Spark, обрабатывающего данные микро-батчами, Apache Flink обрабатывает каждое входящее событие индивидуально.

В Apache Flink механизм Event Time задается декларативно при создании DDL: `WATERMARK FOR ts AS ts - INTERVAL '2' SECOND`. Flink использует водяные метки для отслеживания хода логического времени событий и корректного закрытия окон при поступлении опоздавших сообщений.

1. Tumbling Windows в Flink

Фиксированные окна по 10 секунд без перекрытия:

```
Starting Flink job with tumbling window...
+----+-------------------------+-------------------------+--------------------------------+----------------------+
| op |               win_start |                 win_end |                      sensor_id |          event_count |
+----+-------------------------+-------------------------+--------------------------------+----------------------+
| +I | 1970-01-01 00:00:00.000 | 1970-01-01 00:00:10.000 |                       sensor_1 |                 1017 |
| +I | 1970-01-01 00:00:00.000 | 1970-01-01 00:00:10.000 |                       sensor_3 |                 1022 |
| +I | 1970-01-01 00:00:00.000 | 1970-01-01 00:00:10.000 |                       sensor_2 |                 1593 |
| +I | 2026-10-27 13:51:10.000 | 2026-10-27 13:51:20.000 |                       sensor_2 |                    9 |
| +I | 2026-10-27 13:51:20.000 | 2026-10-27 13:51:30.000 |                       sensor_2 |                    8 |
| +I | 2026-10-27 13:51:30.000 | 2026-10-27 13:51:40.000 |                       sensor_2 |                    4 |
| +I | 2026-10-27 13:51:40.000 | 2026-10-27 13:51:50.000 |                       sensor_2 |                    7 |
| +I | 2026-10-27 13:51:50.000 | 2026-10-27 13:52:00.000 |                       sensor_2 |                    8 |
| +I | 2026-10-27 13:52:00.000 | 2026-10-27 13:52:10.000 |                       sensor_2 |                    9 |
| +I | 2026-10-27 13:52:10.000 | 2026-10-27 13:52:20.000 |                       sensor_2 |                    8 |
| +I | 2026-10-27 13:52:20.000 | 2026-10-27 13:52:30.000 |                       sensor_2 |                    8 |
| +I | 2026-10-27 13:52:30.000 | 2026-10-27 13:52:40.000 |                       sensor_2 |                    7 |
| +I | 2026-10-27 13:52:40.000 | 2026-10-27 13:52:50.000 |                       sensor_2 |                    7 |
| +I | 2026-10-27 13:52:50.000 | 2026-10-27 13:53:00.000 |                       sensor_2 |                    7 |
```

Окна строго разделены: `13:51:10-13:51:20`, `13:51:20-13:51:30`. Каждое сообщение инкрементирует счетчик `event_count` строго в своем временном интервале.

2. Sliding (HOP) Windows в Flink

Окна длиной 10 секунд со сдвигом каждые 5 секунд:

```
Starting Flink job with sliding window...
+----+-------------------------+-------------------------+--------------------------------+----------------------+
| op |               win_start |                 win_end |                      sensor_id |          event_count |
+----+-------------------------+-------------------------+--------------------------------+----------------------+
| +I | 1969-12-31 23:59:55.000 | 1970-01-01 00:00:05.000 |                       sensor_3 |                 1540 |
| +I | 1970-01-01 00:00:00.000 | 1970-01-01 00:00:10.000 |                       sensor_3 |                 1540 |
| +I | 2026-10-27 13:51:05.000 | 2026-10-27 13:51:15.000 |                       sensor_3 |                    3 |
| +I | 2026-10-27 13:51:10.000 | 2026-10-27 13:51:20.000 |                       sensor_3 |                    4 |
| +I | 2026-10-27 13:51:15.000 | 2026-10-27 13:51:25.000 |                       sensor_3 |                    6 |
| +I | 2026-10-27 13:51:20.000 | 2026-10-27 13:51:30.000 |                       sensor_3 |                    7 |
| +I | 2026-10-27 13:51:25.000 | 2026-10-27 13:51:35.000 |                       sensor_3 |                    7 |
| +I | 2026-10-27 13:51:30.000 | 2026-10-27 13:51:40.000 |                       sensor_3 |                   10 |
| +I | 2026-10-27 13:51:35.000 | 2026-10-27 13:51:45.000 |                       sensor_3 |                    7 |
| +I | 2026-10-27 13:51:40.000 | 2026-10-27 13:51:50.000 |                       sensor_3 |                    4 |
| +I | 2026-10-27 13:51:45.000 | 2026-10-27 13:51:55.000 |                       sensor_3 |                    4 |
| +I | 2026-10-27 13:51:50.000 | 2026-10-27 13:52:00.000 |                       sensor_3 |                    8 |
| +I | 2026-10-27 13:51:55.000 | 2026-10-27 13:52:05.000 |                       sensor_3 |                   10 |
| +I | 2026-10-27 13:52:00.000 | 2026-10-27 13:52:10.000 |                       sensor_3 |                    6 |
| +I | 2026-10-27 13:52:05.000 | 2026-10-27 13:52:15.000 |                       sensor_3 |                    6 |
| +I | 2026-10-27 13:52:10.000 | 2026-10-27 13:52:20.000 |                       sensor_3 |                    7 |
```

Зафиксировано перекрытие: интервал `13:51:05-13:51:15` и следующий за ним `13:51:10-13:51:20` имеют общую 5-секундную область, в которой сообщения дублируются в агрегациях обоих окон.

3. Session Windows в Flink

Динамические окна сессий с таймаутом неактивности 10 секунд:

```
Starting Flink job with session window...
+----+-------------------------+-------------------------+--------------------------------+----------------------+
| op |               win_start |                 win_end |                      sensor_id |          event_count |
+----+-------------------------+-------------------------+--------------------------------+----------------------+
| +I | 1970-01-01 00:00:00.000 | 1970-01-01 00:00:10.000 |                       sensor_3 |                 1540 |
| +I | 2026-10-27 13:51:10.513 | 2026-10-27 14:04:53.877 |                       sensor_3 |                  539 |
| +I | 1970-01-01 00:00:00.000 | 1970-01-01 00:00:10.000 |                       sensor_2 |                 1593 |
| +I | 2026-10-27 13:51:11.621 | 2026-10-27 14:01:54.866 |                       sensor_2 |                  412 |
| +I | 1970-01-01 00:00:00.000 | 1970-01-01 00:00:10.000 |                       sensor_1 |                 1510 |
| +I | 2026-10-27 14:04:54.912 | 2026-10-27 14:10:57.016 |                       sensor_3 |                  231 |
```

Начало и конец окон имеют миллисекундную точность и адаптируются под непрерывный поток данных: сессия для `sensor_3` непрерывно длилась с `13:51:10.513` по `14:04:53.877` и накопила 539 событий до наступления паузы.

---

7. Сравнительный анализ задержек и пропускной способности

В ходе экспериментов над потоком из Kafka были измерены ключевые метрики производительности Spark Structured Streaming и Apache Flink:

Метрика	Apache Spark Structured Streaming	Apache Flink
Архитектурный подход	**Micro-batching** (микро-пакеты). Данные накапливаются в течение интервала триггера и обрабатываются пачкой.	**True Streaming**. Каждое событие обрабатывается индивидуально в момент прихода.
End-to-End Latency	**Высокая (~10–11 секунд).** В логах Spark было зафиксировано предупреждение: *«The trigger interval is 10000 ms, but spent 11164 ms»*. Минимальная задержка жестко ограничена размером микро-батча.	**Сверхнизкая (единицы миллисекунд, ~10–50 мс).** Событие сразу проходит через конвейер операторов без ожидания накопления пачки.
Throughput (Пропускная способность)	**Экстремально высокая на больших объемах.** За счет пакетной векторизации Spark отлично утилизирует процессорные кэши и память при миллионных объемах строк/сек.	**Высокая.** Обеспечивает стабильный темп обработки даже при пиковых нагрузках без скачков задержки за счет внутреннего механизма Backpressure.
Управление состоянием (State Management)	State Store в памяти микро-батча. Очищается только по водяным меткам (Watermarks).	Встроенные бэкенды состояния с инкрементальными чекпоинтами.
Сеансовые окна	Ограничены: требуют режима вывода `Complete` (обновления не поддерживаются).	Нативная поддержка сессий через DataStream и SQL API.